BÀI THỰC HÀNH 5. LẬP TRÌNH KỊCH BẢN
KIỂM THỬ HỆ THỐNG

III. Củng cố

Api_server.py:

```
==========================================================================
import os
import sqlite3
from flask import Flask, jsonify, request

app = Flask(__name__)
DB_FILE = "tasks.db"
ADMIN_TOKEN = "SECRET_ADMIN_TOKEN"


def get_db_connection():
    conn = sqlite3.connect(DB_FILE)
    conn.row_factory = sqlite3.Row
    return conn


def init_db():
    """Tạo bảng tasks nếu CSDL chưa tồn tại."""
    conn = get_db_connection()
    conn.execute(
        """
        CREATE TABLE IF NOT EXISTS tasks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            completed BOOLEAN NOT NULL CHECK (completed IN (0, 1))
        )
    """
    )
    conn.commit()
    conn.close()


# API 1: Lấy danh sách tất cả công việc (GET /tasks)
@app.route("/tasks", methods=["GET"])
def get_tasks():
    conn = get_db_connection()
    tasks_cursor = conn.execute("SELECT * FROM tasks").fetchall()
    conn.close()
    tasks = [dict(row) for row in tasks_cursor]
    for t in tasks:
        t["completed"] = bool(t["completed"])
    return jsonify(tasks), 200


# API 2: Tạo công việc mới (POST /tasks)
@app.route("/tasks", methods=["POST"])
```

```
def add_task():
    new_task = request.get_json()
    if not new_task or "title" not in new_task:
        return jsonify({"error": "Missing title"}), 400

    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO tasks (title, completed) VALUES (?, ?)",
        (new_task["title"], False),
    )
    new_id = cursor.lastrowid
    conn.commit()
    conn.close()
    return (
        jsonify({"id": new_id, "title": new_task["title"], "completed": False}),
        201,
    )


# API 3: Lấy chi tiết công việc theo ID (GET /tasks/<id>)
@app.route("/tasks/<int:task_id>", methods=["GET"])
def get_task(task_id):
    conn = get_db_connection()
    task_row = conn.execute(
        "SELECT * FROM tasks WHERE id = ?", (task_id,)
    ).fetchone()
    conn.close()

    if task_row is None:
        return jsonify({"error": "Task not found"}), 404

    task = dict(task_row)
    task["completed"] = bool(task["completed"])
    return jsonify(task), 200


# API 4: Cập nhật công việc (PUT /tasks/<id>)
@app.route("/tasks/<int:task_id>", methods=["PUT"])
def update_task(task_id):
    data = request.get_json()
    if not data:
        return jsonify({"error": "Invalid payload"}), 400

    conn = get_db_connection()
    task_row = conn.execute(
        "SELECT * FROM tasks WHERE id = ?", (task_id,)
    ).fetchone()

    if task_row is None:
        conn.close()
        return jsonify({"error": "Task not found"}), 404

    new_title = data.get("title", task_row["title"])
```

```
    new_completed = data.get("completed", task_row["completed"])

    conn.execute(
        "UPDATE tasks SET title = ?, completed = ? WHERE id = ?",
        (new_title, int(new_completed), task_id),
    )
    conn.commit()
    conn.close()

    return (
        jsonify(
            {"id": task_id, "title": new_title, "completed": bool(new_completed)}
        ),
        200,
    )


# API 5: Xóa công việc (DELETE /tasks/<id>)
@app.route("/tasks/<int:task_id>", methods=["DELETE"])
def delete_task(task_id):
    conn = get_db_connection()
    cursor = conn.execute("DELETE FROM tasks WHERE id = ?", (task_id,))
    conn.commit()
    conn.close()

    if cursor.rowcount == 0:
        return jsonify({"error": "Task not found"}), 404

    return jsonify({"message": "Task deleted successfully"}), 200


# API 6: Xóa toàn bộ dữ liệu - Dành cho Admin (POST /admin/clear_all)
@app.route("/admin/clear_all", methods=["POST"])
def clear_all():
    token = request.headers.get("X-Admin-Token")
    if token != ADMIN_TOKEN:
        return jsonify({"error": "Unauthorized"}), 401

    conn = get_db_connection()
    conn.execute("DELETE FROM tasks")
    conn.commit()
    conn.close()

    return jsonify({"message": "Database cleared successfully"}), 200


if __name__ == "__main__":
    init_db()
    app.run(host="127.0.0.1", port=5000, debug=True)
```

Test_system.py:

```
==========================================================================
import os
```

```
import threading
import unittest
import requests

BASE_URL = "http://127.0.0.1:5000"
ADMIN_TOKEN = "SECRET_ADMIN_TOKEN"


class TestSystem(unittest.TestCase):

    def reset_database_via_api(self):
        """Xóa sạch CSDL bằng API Admin trước mỗi bài kiểm thử."""
        headers = {"X-Admin-Token": ADMIN_TOKEN}
        requests.post(f"{BASE_URL}/admin/clear_all", headers=headers)

    # ======================================================================
    # BÀI 1: KIỂM THỬ CHỨC NĂNG HỆ THỐNG (Functional System Testing)
    # ======================================================================
    def test_01_full_user_workflow(self):
        print("\n--- Running Bài 1: test_full_user_workflow ---")
        self.reset_database_via_api()

        # 1. Tạo công việc mới "Học bài Kiểm thử"
        res_create = requests.post(
            f"{BASE_URL}/tasks", json={"title": "Học bài Kiểm thử"}
        )
        self.assertEqual(res_create.status_code, 201)
        created_task = res_create.json()
        self.assertEqual(created_task["title"], "Học bài Kiểm thử")
        self.assertFalse(created_task["completed"])
        task_id = created_task["id"]

        # 2. Kiểm tra công việc có trong danh sách tất cả công việc
        res_get_all = requests.get(f"{BASE_URL}/tasks")
        self.assertEqual(res_get_all.status_code, 200)
        tasks_list = res_get_all.json()
        self.assertTrue(any(t["id"] == task_id for t in tasks_list))

        # 3. Cập nhật thành "Đã học xong" và completed = True
        update_payload = {"title": "Đã học xong", "completed": True}
        res_update = requests.put(
            f"{BASE_URL}/tasks/{task_id}", json=update_payload
        )
        self.assertEqual(res_update.status_code, 200)

        # 4. Kiểm tra lại thông tin sau cập nhật
        res_get_single = requests.get(f"{BASE_URL}/tasks/{task_id}")
        self.assertEqual(res_get_single.status_code, 200)
        updated_task = res_get_single.json()
        self.assertEqual(updated_task["title"], "Đã học xong")
        self.assertTrue(updated_task["completed"])

        # 5. Xóa công việc
        res_delete = requests.delete(f"{BASE_URL}/tasks/{task_id}")
```

```
        self.assertEqual(res_delete.status_code, 200)

        # 6. Kiểm tra lại danh sách rỗng
        res_final = requests.get(f"{BASE_URL}/tasks")
        self.assertEqual(res_final.status_code, 200)
        self.assertEqual(len(res_final.json()), 0)

        print("=> PASS: Bài 1 đã hoàn thành luồng nghiệp vụ người dùng.")

    # ==========================================================================
    # BÀI 2: KIỂM THỬ CHỊU TẢI (Load Testing)
    # ==========================================================================
    def test_02_load_handling(self):
        print("\n--- Running Bài 2: test_load_handling ---")
        thread_count = 50
        results = []

        def worker():
            try:
                response = requests.get(f"{BASE_URL}/tasks")
                results.append(response.status_code)
            except requests.exceptions.RequestException as e:
                results.append(str(e))

        # Tạo 50 luồng đồng thời
        threads = [
            threading.Thread(target=worker) for _ in range(thread_count)
        ]
        for t in threads:
            t.start()
        for t in threads:
            t.join()

        # Kiểm tra kết quả
        self.assertEqual(len(results), thread_count)
        for status_code in results:
            self.assertEqual(status_code, 200)

        print(
            f"=> PASS: Bài 2 đã xử lý thành công {thread_count} yêu cầu đồng thời."
        )


# ==========================================================================
    # BÀI 3: KIỂM THỬ HỒI PHỤC (Recovery Testing)
    # ==========================================================================
    def test_03_recovery_verify_after_restart(self):
        print("\n--- Running Bài 3: test_recovery_verify_after_restart ---")
        # Reset và tự động tạo sẵn 3 công việc trước khi kiểm tra
        self.reset_database_via_api()
        for i in range(1, 4):
            requests.post(f"{BASE_URL}/tasks", json={'title': f'Công việc {i}'})

        # Đọc danh sách công việc từ server
```

```
        res = requests.get(f"{BASE_URL}/tasks")
        self.assertEqual(res.status_code, 200)
        tasks = res.json()

        # Kiểm tra xác minh đủ 3 công việc
        self.assertEqual(
            len(tasks),
            3,
            "Số lượng công việc không đúng! Cần 3 công việc."
        )
        print("=&gt; PASS: Bài 3 khôi phục và xác minh 3 công việc thành công!")


    # ================================================================================
    # BÀI 4: KIỂM THỬ AN TOÀN (Security Testing)
    # ================================================================================
    def test_04_security_admin_endpoint(self):
        print("\n--- Running Bài 4: test_security_admin_endpoint ---")

        # 1. Không có header X-Admin-Token -> 401 Unauthorized
        res_no_token = requests.post(f"{BASE_URL}/admin/clear_all")
        self.assertEqual(res_no_token.status_code, 401)

        # 2. X-Admin-Token sai -> 401 Unauthorized
        res_wrong_token = requests.post(
            f"{BASE_URL}/admin/clear_all",
            headers={"X-Admin-Token": "wrong_token"},
        )
        self.assertEqual(res_wrong_token.status_code, 401)

        # 3. X-Admin-Token đúng -> 200 OK
        res_valid_token = requests.post(
            f"{BASE_URL}/admin/clear_all",
            headers={"X-Admin-Token": ADMIN_TOKEN},
        )
        self.assertEqual(res_valid_token.status_code, 200)

        print("=> PASS: Bài 4 kiểm thử an toàn API Admin đạt yêu cầu.")


if __name__ == "__main__":
    unittest.main(argv=["first-arg-is-ignored"], exit=False)
```